Практическая работа №2

Знакомство с Dart и Flutter. ООП.

Целью второй практической работы является знакомство с основными подходами к разработке программного обеспечения - объектно-ориентированным и функциональным.

ООП характерно для языков вроде Java или C#. Основной категорией такого подхода является класс.

Функциональное программирование больше используется в JS и Python. Основной категорией этого подхода является функция.

Для продолжения необходимо рассмотреть следующие разделы документации:

Функция: https://metanit.com/dart/tutorial/3.1.php

Области видимости: https://metanit.com/dart/tutorial/3.7.php

Параметры функции: https://metanit.com/dart/tutorial/3.2.php

Функция - объект: https://metanit.com/dart/tutorial/3.4.php

Лямбда-функция: https://metanit.com/dart/tutorial/3.5.php

Классы: https://metanit.com/dart/tutorial/4.1.php

Конструкторы: https://metanit.com/dart/tutorial/4.12.php

Инкапсуляция: https://metanit.com/dart/tutorial/4.13.php

Null: https://metanit.com/dart/tutorial/4.16.php

Константы: https://metanit.com/dart/tutorial/4.2.php

Статика: https://metanit.com/dart/tutorial/4.7.php

Наследование: https://metanit.com/dart/tutorial/4.3.php

Преобразования типов: https://metanit.com/dart/tutorial/4.17.php

Абстрактные классы: https://metanit.com/dart/tutorial/4.4.php

Задание:

1. Функции высшего порядка + ООП

Цель: Написать класс MathProcessor, который:

1. Принимает в конструкторе список чисел List<double>.
2. Имеет метод process, принимающий функцию-обработчик (например, возведение в квадрат) и возвращающий новый список.
3. Реализует статический метод getRandomNumbers, генерирующий список случайных чисел.

2. Абстрактные классы + Наследование

Цель: Реализовать иерархию классов для геометрических фигур:

1. **Абстрактный класс** Shape с методом area().
2. Классы Circle и Rectangle, наследующие Shape.
3. **Метод** Shap.fromJson(Map<String, dynamic> json), который создает Circle или Rectangle на основе данных JSON.

Условия:

Для Circle JSON: {"type": "circle", "radius": 5}.

Для Rectangle JSON: {"type": "rectangle", "width": 4, "height": 6}.

3. Статические методы + Константы

Цель: Реализовать класс PasswordValidator с:

1. Константными требованиями к паролю: Минимум 8 символов. Хотя бы одна цифра.
2. Статическим методом validate, проверяющим пароль.

4. Банковская система

Условие:

- Реализуйте систему банковских счетов с использованием ООП. Требуется:
- **Класс BankAccount**
  - Поля:
    - accountNumber (уникальный номер счета)
    - balance (текущий баланс, начальное значение = 0)
    - ownerName (имя владельца)
  - Методы:
    - deposit(amount) – пополнение счета (нельзя внести отрицательную сумму).
    - withdraw(amount) – снятие денег (нельзя снять больше, чем есть на счету).
    - displayBalance() – вывод текущего баланса.
  - **Дополнительно:**
    - При создании счета генерировать случайный accountNumber (например, из 10 цифр).
- **Класс Bank**
  - Поля:
    - List<BankAccount> accounts (список всех счетов).
  - Методы:
    - addAccount(ownerName) – создает новый счет и добавляет его в accounts.
    - findAccount(accountNumber) – возвращает счет по номеру (или null, если не найден).
    - transfer(fromAccountNumber, toAccountNumber, amount) – перевод денег между счетами.

5. Игра «РПГ-персонажи»

Условие:

Создайте систему классов для RPG-персонажей с использованием абстрактных классов и интерфейсов.

1. **Абстрактный класс Character**
   a. Поля:
      i. name (имя персонажа)
      ii. health (здоровье, начальное = 100)
      iii. level (уровень, начальное = 1)

b. Абстрактные методы:
    i. attack() – возвращает урон.
    ii. defend(damage) – уменьшает здоровье на damage.
c. Общие методы:
    i. isAlive() – проверяет, жив ли персонаж (health > 0).

2. **Классы персонажей:**
    a. **Warrior**:
        i. Атака: 15 + level * 2.
        ii. Защита: уменьшает урон на 20%.
    b. **Mage**:
        i. Атака: 10 + level * 3.
        ii. Защита: получает полный урон, но может вылечиться на 5 HP после атаки.
    c. **Rogue**:
        i. Атака: 20% шанс на критический удар (x3).
        ii. Защита: уворот (50% шанс избежать урона).

Дополнительно:

d. Реализуйте бой между двумя персонажами (по очереди атакуют друг друга).

Формат отчетности: pdf файл, в названии: имя. фамилия, группа, номер работы.